

Cross-Engine Learned Cost Models for SQL Workloads

Rinilkumar Parmar
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
parmar7f@uwindsor.ca

Aditiben Fatesinh Parmar
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
parmar3f@uwindsor.ca

Lisa Vijaybhai Mangnani
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
mangnan@uwindsor.ca

Krishal Modi
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
modi3b@uwindsor.ca

Janitha Reddy Salla
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
salla@uwindsor.ca

Krish Dimplekumar Thakkar
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
thakka4b@uwindsor.ca

Hiten Jagdishbhai Patil
Dept. of Computer Engineering
University of Windsor
Windsor, ON, Canada
patil46@uwindsor.ca

Abstract—Modern database deployments increasingly adopt composable architectures, where the same SQL workload can be executed by multiple heterogeneous database engines such as SQLite, DuckDB, or PostgreSQL. Despite this shift, existing query optimisers are inherently engine-local and provide no principled mechanism for selecting the most suitable execution engine for a given query. Consequently, engine selection is commonly performed using manual heuristics, offline benchmarking, or trial execution, which leads to suboptimal performance and operational complexity.

In this work, we propose a cross-engine learned cost model that predicts SQL query execution time across multiple database engines using features derived from logical query plans. Our approach enables zero-shot engine selection, allowing the system to choose the fastest execution engine prior to query execution without modifying database internals. We emphasise a simple and reproducible design based on lightweight machine-learning models and evaluate the system based on engine-selection accuracy rather than cost prediction error alone. Experimental results on standard analytical workloads demonstrate that the proposed approach consistently selects the optimal execution engine and outperforms heuristic-based baselines. These findings suggest that extending cost-based optimisation beyond single-engine boundaries is both feasible and beneficial for modern database systems.

Index Terms—Query optimisation, learned cost models, multi-engine databases, SQL workloads, database systems, and machine learning for databases

I. INTRODUCTION AND MOTIVATION

Database systems are undergoing a fundamental shift from monolithic deployments to composable architectures, where query planning, optimisation, and execution may involve multiple independent database engines. Lightweight embedded

engines such as SQLite are widely used for transactional and resource-constrained workloads, while analytical engines such as DuckDB are increasingly adopted for vectorized, in-memory analytics. In modern data stacks, it is increasingly common for the same SQL query to be executable by more than one engine with vastly different performance characteristics.

Despite this architectural shift, query optimisation remains largely engine-local. Existing database optimisers are designed under the assumption that a query will be executed within a single DBMS, and their cost models are calibrated to compare alternative execution plans inside that engine. When multiple engines are available, the decision of which engine should execute a given query is typically handled outside the optimiser using manual heuristics, static rules, or offline benchmarking. These approaches are brittle, workload-specific, and often fail to generalise as data characteristics and query patterns evolve.

This limitation is increasingly problematic for modern SQL workloads. Analytical queries with joins and aggregations may benefit significantly from vectorised execution and aggressive in-memory processing, while simpler queries may incur unnecessary overhead when executed on heavier engines. Without a principled mechanism for engine selection, systems may consistently choose suboptimal execution paths, leading to increased query latency, inefficient resource utilisation, and higher operational complexity. Moreover, trial execution or query duplication across engines introduces additional overhead that undermines performance gains.

Recent advances in learned cost models have shown promise in improving query optimisation by predicting execution cost

using machine-learning techniques. Prior work demonstrates that learned models can outperform traditional hand-crafted cost estimators for tasks such as cardinality estimation and plan selection. However, this line of research has almost exclusively focused on optimisation within a single database engine. The broader problem of using learned cost models to reason across heterogeneous engines remains underexplored.

This project is motivated by the observation that logical query structure alone encodes substantial information about execution complexity, independent of the underlying engine. Features such as join patterns, predicate structure, and aggregation depth can be leveraged to predict relative performance across engines without requiring access to physical execution plans or engine internals. By learning engine-specific performance characteristics from historical executions, it becomes possible to select the most suitable engine before query execution.

Addressing this problem is both timely and important. As database systems continue to diversify and composability becomes the norm rather than the exception, the lack of cross-engine optimisation represents a growing gap in current database technology. A practical, cost-based approach to engine selection can significantly improve performance while preserving the declarative abstraction that lies at the core of database systems. This work aims to take a first step toward filling this gap by designing and evaluating a cross-engine learned cost model that enables zero-shot SQL query engine selection in modern database environments.

II. PROBLEM STATEMENT

Modern database deployments increasingly provide access to multiple SQL execution engines, each optimised for different workload characteristics. Engines such as SQLite, DuckDB, and PostgreSQL differ substantially in their execution models, memory usage, and parallelisation strategies, yet they are often capable of executing the same logical SQL queries. In such environments, selecting the most appropriate execution engine for a given query has a direct impact on performance, resource utilisation, and system efficiency.

Current database systems lack a principled mechanism for making this decision. Query optimisers are designed to operate within a single engine and rely on engine-specific cost models that compare alternative execution plans under the assumption that the engine itself is fixed. When multiple engines are available, engine selection is typically handled outside the optimiser using static heuristics, manual tuning, or offline benchmarking. These approaches are brittle, workload-dependent, and unable to adapt to changing query patterns or data distributions.

Furthermore, traditional cost models are poorly suited to compare performance across heterogeneous engines. They rely on hand-crafted formulas and assumptions about operator behaviour that do not generalise across execution paradigms such as row-based versus vectorised processing. As a result, even accurate intra-engine cost estimates provide little guidance for cross-engine decisions. Executing queries on

multiple engines to empirically determine the fastest option introduces additional overhead and undermines the benefits of optimisation.

The core problem addressed in this project is the absence of a cross-engine cost-based mechanism that can predict query execution time across multiple database engines prior to execution. Specifically, given a SQL query and a set of available execution engines, the system must estimate the relative execution cost of the query on each engine using only information available before execution. Solving this problem enables zero-shot engine selection, allowing the system to choose the most suitable engine without query execution, manual intervention, or engine-specific tuning.

This problem is significant because it sits at the intersection of cost-based optimisation, system composability, and practical database deployment. Addressing it requires extending classical optimisation principles beyond single-engine boundaries while preserving the declarative abstraction that underpins database systems. The objective of this project is therefore to design, implement, and evaluate a learned cost model that enables accurate and practical cross-engine engine selection for SQL workloads.

III. RELATED WORK

The closest related work to this project is A Learned Cost Model-based Cross-engine Optimiser for SQL Workloads, which introduces a graph neural network with a multi-head architecture to predict query execution time across multiple database engines. This work demonstrates that cross-engine cost prediction is feasible and can outperform heuristic engine selection. However, its primary focus is on sophisticated model architecture and controlled experimental evaluation. In contrast, our work is not a reproduction: we emphasise a simple and reproducible baseline, prioritise engine-selection accuracy and failure analysis rather than prediction error alone, and target practical feasibility in composable systems without optimiser-internal integration. The GNN-based approach is treated as an optional extension rather than a prerequisite.

Several recent studies motivate learned cost modeling but do not address cross-engine optimisation. Learned Cost Models for Query Optimisation: From Batch to Streaming Systems surveys learned cost models and highlights their advantages over traditional estimators, but assumes a single execution engine and does not consider engine selection. How Good Are Learned Cost Models, Really? provides a critical evaluation of learned cost models for plan selection, yet limits its scope to intra-engine optimisation and does not explore heterogeneous engine routing.

Prior work on plan-based learning focuses on improving optimisation decisions within a single DBMS. Plan-Structured Deep Neural Network Models for Query Performance Prediction demonstrates that execution time can be predicted from plan structure, but does not extend this reasoning across engines. Similarly, Redefining Cost Estimation in Database Systems studies execution-time prediction using plan features,

but does not evaluate multi-engine environments or engine selection decisions.

Research on big-data and distributed engines further illustrates the limitations of engine-specific cost models. A Learned Cost Model for Big Data Query Processing targets Spark-like systems and incorporates resource modelling, but assumes a fixed execution engine. Cost Models for Big Data Query Processing: retrofit learned models into traditional optimisers, yet remain engine-local. Earlier work, such as Predicting Query Execution Time: Are Optimiser Cost Models Sufficient? motivates learning-based prediction, but predates modern vectorised execution and composable database architectures.

Gap Summary. Existing research either improves cost estimation within a single engine or redesigns database architectures. None evaluate a simple, practical cross-engine learned cost model that enables zero-shot engine selection in composable SQL systems while explicitly analysing robustness and failure cases. This gap motivates our work.

IV. NOVELTY CONTRIBUTIONS

The novelty of this work lies in extending learned cost modelling beyond traditional intra-engine query optimisation to the problem of cross-engine SQL query execution. Existing database optimisers and learned cost models are designed under the assumption that a query will be executed within a single database engine, and that cost estimation is required only to compare alternative execution plans inside that engine. In contrast, this work treats engine selection itself as a cost-based optimisation problem, addressing a growing gap in modern composable database environments.

The central assumption challenged by this work is that engine selection must rely on manual heuristics, offline benchmarking, or empirical trial execution. We demonstrate that relative query execution performance across heterogeneous engines can be predicted prior to execution using simple features derived from logical query plans. By avoiding dependence on physical plans, engine internals, or optimiser modifications, the proposed approach remains engine-agnostic and practical to deploy.

Compared to prior research on learned cost models, which primarily focuses on improving plan selection accuracy within a single DBMS, our work shifts the optimisation scope to a higher abstraction level. We show that lightweight machine-learning models are sufficient to capture systematic performance differences between engines such as SQLite and DuckDB, establishing a strong and reproducible baseline for cross-engine optimisation.

The contributions of this work are therefore focused and concrete. First, we design a cross-engine cost prediction framework that enables zero-shot engine selection using only pre-execution information. Second, we evaluate the approach using engine-selection accuracy as the primary metric, directly reflecting end-to-end system benefit rather than prediction error alone. Finally, we outline a clear extension path toward more expressive models, such as graph-based representations, while maintaining a simple and robust core design.

V. SYSTEM DESIGN / ALGORITHM OVERVIEW

The proposed system is an external, engine-agnostic decision layer that selects the most suitable SQL execution engine for a query prior to execution. The design deliberately avoids any modification to database internals or query optimizers and relies only on information available before query execution, ensuring portability and ease of deployment.

Given a SQL query, the system first extracts a logical query plan using the EXPLAIN interface of each supported engine. These plans describe the structural composition of the query, including scans, joins, filters, and aggregations, without exposing engine-specific physical execution details. Each logical plan is then converted into a compact, fixed-length feature representation capturing basic structural characteristics such as the number of joins, predicates, aggregation operators, and overall plan depth.

A learned cost model is applied to these features to predict query execution time for each candidate engine. In the base design, lightweight regression models are used to estimate engine-specific execution cost, balancing accuracy and simplicity. The system then selects the engine with the lowest predicted cost and executes the query using that engine's native optimizer and execution pipeline.

This workflow enables zero-shot engine selection, as decisions are made entirely prior to execution. The architecture is intentionally modular, allowing feature-based models to be replaced with more expressive representations, such as graph-structured plans and neural models, without altering the overall system flow.

VI. IMPLEMENTATION PLAN

The proposed system will be implemented as an external decision layer that operates independently of database internals. The implementation will be carried out in Python due to its strong ecosystem for database connectivity, data processing, and machine-learning libraries. Two database engines, SQLite and DuckDB, will be used as the initial execution backends to ensure a manageable scope while capturing meaningful differences in execution behaviour.

Query workloads will be derived from standard analytical benchmarks, primarily TPC-H, along with synthetically generated query variants to increase workload diversity. Queries will be executed on both engines, and execution time will be recorded to serve as ground-truth cost labels. Logical query plans will be extracted using the EXPLAIN interface provided by each engine. These plans will be parsed to derive engine-agnostic structural features such as the number of joins, selection predicates, aggregation operators, and overall plan depth.

Feature extraction and data preprocessing will be implemented as modular components to facilitate extensibility. Learned cost models will be trained using lightweight machine-learning techniques, such as linear regression or random forest regressors, implemented with scikit-learn. Separate cost models will be maintained for each engine to capture

engine-specific performance characteristics while sharing a common feature representation.

The system will include a runtime engine-selection module that applies the trained cost models to incoming queries and selects the execution engine with the lowest predicted cost. The selected engine will then execute the query using its native optimiser and execution pipeline. No modifications to database engines or query optimisers will be required.

The implementation is designed to support incremental extensions. While the initial system uses feature-based representations and simple models, the architecture allows for replacing feature vectors with graph-structured plan representations and introducing more expressive learning models, such as graph neural networks, without changing the overall execution flow.

VII. RISKS MITIGATION

A primary technical risk is that simple, feature-based cost models may yield insufficient prediction accuracy for certain query patterns, particularly for queries with complex joins or skewed data distributions. To mitigate this risk, we will iteratively refine the feature set and evaluate multiple lightweight regression models. Additionally, performance will be assessed using engine-selection accuracy rather than prediction error alone, ensuring that modest estimation errors do not undermine the primary optimisation goal.

A second risk concerns limited generalisation across workloads. Cost models trained on a narrow query distribution may overfit and perform poorly on unseen queries. This risk will be addressed by using diverse query workloads derived from standard benchmarks and synthetically generated variants, as well as by employing cross-validation during training and evaluation.

Measurement noise and variability in execution time represent another experimental risk, especially for short-running queries. To mitigate this, queries will be executed multiple times, and the median execution time will be used as the ground-truth label. This reduces the impact of transient system noise and improves result stability.

Finally, there is a risk of scope creep if additional engines or complex models are introduced prematurely. This will be mitigated by restricting the initial implementation to two engines and lightweight models, with more advanced techniques treated strictly as optional extensions. This staged approach ensures feasibility while preserving a clear path for future enhancement.

VIII. TIMELINE MILESTONES

The project will be executed in clearly defined phases to ensure steady progress and timely delivery. The initial phase focuses on environment setup, dataset preparation, and workload selection. During this phase, the target database engines will be configured, benchmark datasets will be loaded, and a representative set of SQL queries will be finalised. The outcome of this phase is a stable experimental setup and a curated query workload.

The second phase centers on data collection and feature extraction. Logical query plans will be extracted using the EXPLAIN interface, and engine-agnostic structural features will be implemented and validated. Queries will be executed on each engine to collect execution-time measurements, resulting in a labelled dataset suitable for training learned cost models.

In the third phase, lightweight machine-learning models will be trained and evaluated. Multiple regression models will be explored to establish a strong baseline for cross-engine cost prediction. This phase concludes with an initial evaluation of prediction accuracy and engine-selection performance against heuristic baselines.

The Final phase focuses on system integration and robustness analysis. The engine-selection module will be integrated into the end-to-end workflow, and additional experiments will be conducted to analyse failure cases and generalisation across query variants. If time permits, this phase will also explore optional extensions using more expressive models.

REFERENCES

- [1] A. Strausz, N. Pardon, and I. Giurciu, "A Learned Cost Model-based Cross-engine Optimizer for SQL Workloads," *arXiv preprint arXiv:2506.02802*, 2025.
- [2] R. Heinrich, X. Li, and Z. Kaoudi, "Learned Cost Models for Query Optimization: From Batch to Streaming Systems," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 18, no. 12, 2025.
- [3] R. Heinrich, M. Luthra, J. Wehrstein, H. Kornmayer, and C. Binnig, "How Good Are Learned Cost Models, Really? Insights from Query Optimization Tasks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2025.
- [4] R. Marcus, P. Negi, H. Mao, N. Tatbul, and T. Kraska, "Plan-Structured Deep Neural Network Models for Query Performance Prediction," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 12, no. 11, 2019.
- [5] U. Pathak and A. Mankodi, "Redefining Cost Estimation in Database Systems: The Role of Execution Plan Features and Machine Learning," *arXiv preprint arXiv:2510.05612*, 2025.
- [6] Y. Li, "A Learned Cost Model for Big Data Query Processing," *Information Sciences*, vol. 681, pp. 1–15, 2024.
- [7] T. Siddiqui, A. Pirk, J. Ding, and T. Kraska, "Cost Models for Big Data Query Processing: Learning, Retrofitting, and Our Findings," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2020.
- [8] W. Wu, Y. Chi, S. Zhu, and J. Tatemura, "Predicting Query Execution Time: Are Optimizer Cost Models Sufficient?," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2013.